Data Structure and Algorithms LAB

Name: Sania Sajjad Roll No: EE241011
Name: Hamiz Rehan Roll No: EE241001

Complex Engineering Problems (CEP)
Search Engine Indexing & Ranking System

1. Introduction

A search engine is a software system designed to store, organize, and retrieve documents based on user queries. It allows users to search for specific keywords and returns relevant results ranked according to their importance.

Search engines such as Google use efficient data structures like hash tables, trees, and heaps to perform fast indexing and searching.

The purpose of this project is to design and implement a mini search engine using fundamental data structures such as Hash Table, Linked List, Binary Search Tree (BST), and Heap. The system builds an inverted index from documents and retrieves ranked results efficiently.

2. Problem Statement

The objective of this project is to design and analyze a mini search engine capable of:

- Building an inverted index from documents
- Searching documents using keywords
- Ranking documents based on relevance
- Using efficient data structures
- Analyzing time and space complexity

3. Objectives

The main objectives are:

- To understand search engine indexing
- To implement inverted index using hash table
- To use linked list for document storage
- To use BST for efficient word storage
- To use heap for ranking documents
- To analyze algorithm performance

4. System Overview

The system consists of four main components:

1. Document Storage
2. Inverted Index Builder
3. Search Module
4. Ranking Module

5. Data Structures Used

5.1 Hash Table

A hash table is used to store the inverted index.

Structure:

```
Word → Linked List of Document IDs
```

Example:

```
electric → 2 → 1
field → 3 → 1
```

Advantages:

- Fast searching

- Average time complexity: O(1)

Implementation in code:

```
self.index = {}
```

5.2 Linked List

Linked list stores document IDs for each word.

Example:

```
electric → 2 → 1
```

Advantages:

- Dynamic memory allocation
- Efficient insertion

Code:

```
class LinkedList:
```

5.3 Binary Search Tree (BST)

BST stores words in sorted order.

Example:

```
        field
       /     \
   electric  theory
```

Advantages:

- Efficient searching
- Organized storage

Code:

```
class BST:
```

5.4 Heap (Max Heap)

Heap is used to rank documents based on keyword frequency.

Example:

```
Document 2 → Score 2
Document 1 → Score 1
```

Code:

```
class MaxHeap:
```

6. Inverted Index

Definition:

An inverted index maps words to documents containing those words.

Example:

Documents:

Doc1:

```
Electric field theory
```

Doc2:

```
Electric circuits and electric machines
```

Inverted index:

```
electric → 2 → 1
field → 1
theory → 1
circuits → 2
```

7. Algorithm

Algorithm 1: Build Inverted Index

Step 1: Start

Step 2: Read document

Step 3: Split document into words
Step 4: For each word
If word not in hash table
Create linked list
Insert document ID into linked list
Step 5: Insert word into BST
Step 6: Repeat for all documents
Step 7: Stop

Algorithm 2: Search and Rank Documents

Step 1: Start
Step 2: Input keyword
Step 3: Search keyword in hash table
Step 4: Get document list
Step 5: Calculate frequency in each document
Step 6: Insert into heap
Step 7: Extract max from heap
Step 8: Display ranked results
Step 9: Stop

8. Code Explanation

Linked List Code Explanation

```
class ListNode:
    def __init__(self, doc_id):
        self.doc_id = doc_id
        self.next = None
```

This creates a node storing document ID.

```
class LinkedList:
```

Stores list of document IDs.
Function:

```
insert()
```

Adds document ID to list.

Hash Table Explanation

```
self.index = {}
```

This creates inverted index.
Example:

```
electric → Linked List
```

BST Explanation

```
class BST:
```

Stores words in sorted order.
Function:

```
insert_word()
```

Inserts words into tree.

Heap Explanation

```
class MaxHeap:
```

Stores document ranking.
Function:

```
insert(doc_id, score)
```

Stores document relevance.

Function:
`extract_max()`
Returns highest ranked document.

Search Engine Class Explanation

`class SearchEngine:`
Main class controlling system.
Variables:
`self.index`
Stores inverted index.
`self.documents`
Stores document text.
`self.tree`
Stores words in BST.
Function:
`add_document()`
Adds document and updates index.
Function:
`search()`
Searches keyword and ranks documents

9. Program Execution

Documents added:

```
engine.add_document(1, "Electric field theory and applications")
engine.add_document(2, "Electric circuits and electric machines")
engine.add_document(3, "Field theory in electromagnetic systems")
engine.add_document(4, "Electric power systems and electric control")
```

10. Program Output Explanation

When program runs:

```
========= MINI SEARCH ENGINE =========
1. Search Keyword
2. Exit
Enter choice: 1
```

User enters:

```
electric
```

Output:

```
Ranked Search Results:
Document ID: 2 | Relevance Score: 2
Document ID: 4 | Relevance Score: 2
Document ID: 1 | Relevance Score: 1
```

Explanation:
Document 2 contains "electric" twice → score 2
Document 4 contains "electric" twice → score 2
Document 1 contains "electric" once → score 1
Heap ranks documents correctly.

11. Time Complexity Analysis

Let:
N = number of documents
W = words per document
D = matched documents
Index building:

`O(N × W)`

Search:

`O(1)`

Ranking:

`O(D log D)`

BST insertion:

`O(log W)`

12. Space Complexity Analysis

Space required:

`O(N × W)`

Components:

Hash Table → O(W)

Linked List → O(N × W)

BST → O(W)

Heap → O(D)

13. Applications

Used in:

- Search engines
- File searching systems
- Digital libraries
- Database search
- Information retrieval systems

14. Advantages

- Fast searching
- Efficient indexing
- Accurate ranking
- Uses efficient data structures

15. Conclusion

This project successfully implemented a mini search engine using Hash Table, Linked List, BST, and Heap.

The system efficiently built an inverted index, performed keyword search, and ranked documents based on relevance.

This project demonstrates how real search engines work internally.

16. Future Improvements

- Use AVL tree instead of BST
- Add multiple keyword search
- Add graphical interface
- Use real text files
- Improve ranking algorithm

Code:

```
class ListNode:  1 usage
    def __init__(self, doc_id):
        self.doc_id = doc_id
        self.next = None
class LinkedList:  1 usage
    def __init__(self):
        self.head = None
    def insert(self, doc_id):  1 usage (1 dynamic)
        temp = self.head
        while temp is not None:
            if temp.doc_id == doc_id:
                return
            temp = temp.next
        new_node = ListNode(doc_id)
        new_node.next = self.head
        self.head = new_node
    def get_all_docs(self):  1 usage (1 dynamic)
        docs = []
        temp = self.head
        while temp is not None:
            docs.append(temp.doc_id)
            temp = temp.next
        return docs
class BSTNode:  1 usage
    def __init__(self, word):
        self.word = word
        self.left = None
        self.right = None
class BST:  1 usage
    def __init__(self):
        self.root = None
    def insert_word(self, word):  1 usage
        self.root = self.insert(self.root, word)
    def insert(self, node, word):  4 usages (1 dynamic)
        if node is None:
            return BSTNode(word)
        if word < node.word:
            node.left = self.insert(node.left, word)
        elif word > node.word:
            node.right = self.insert(node.right, word)
        return node
class MaxHeap:  1 usage
    def __init__(self):
        self.heap = []
    def insert(self, doc_id, score):  2 usages (1 dynamic)
        self.heap.append([doc_id, score])
        self.heapify_up(len(self.heap) - 1)
    def heapify_up(self, index):  1 usage
        while index > 0:
            parent = (index - 1) // 2
            if self.heap[index][1] > self.heap[parent][1]:
                temp = self.heap[index]
                self.heap[index] = self.heap[parent]
                self.heap[parent] = temp
                index = parent
            else:
                break
    def heapify_down(self, index):  1 usage
        size = len(self.heap)
        while True:
            left = 2 * index + 1
            right = 2 * index + 2
            largest = index
            if left < size and self.heap[left][1] > self.heap[largest][1]:
                largest = left
            if right < size and self.heap[right][1] > self.heap[largest][1]:
                largest = right
            if largest != index:
                temp = self.heap[index]
                self.heap[index] = self.heap[largest]
                self.heap[largest] = temp
                index = largest
            else:
                break
    def extract_max(self):  1 usage
        if len(self.heap) == 0:
            return None
        if len(self.heap) == 1:
            return self.heap.pop()
        root = self.heap[0]
        self.heap[0] = self.heap.pop()
        self.heapify_down(0)
        return root
class SearchEngine:  1 usage
    def __init__(self):
        self.index = {}
        self.documents = {}
        self.tree = BST()
    def add_document(self, doc_id, text):  4 usages
        self.documents[doc_id] = text
        words = text.lower().split()
        for word in words:
            if word not in self.index:
                self.index[word] = LinkedList()
                self.tree.insert_word(word)
            self.index[word].insert(doc_id)
    def search(self, keyword):  1 usage
        keyword = keyword.lower()
        if keyword not in self.index:
            print("\nNo documents found.")
            return
        doc_list = self.index[keyword].get_all_docs()
        heap = MaxHeap()
        for doc_id in doc_list:
            text = self.documents[doc_id]
            words = text.lower().split()
            score = 0
            for w in words:
                if w == keyword:
                    score = score + 1
            heap.insert(doc_id, score)
        print("\nRanked Search Results:")
        while True:
            result = heap.extract_max()
            if result is None:
                break
            print("Document ID:", result[0], "| Relevance Score:", result[1])
```

```
engine = SearchEngine()
engine.add_document( doc_id: 1,  text: "Electric field theory and applications")
engine.add_document( doc_id: 2,  text: "Electric circuits and electric machines")
engine.add_document( doc_id: 3,  text: "Field theory in electromagnetic systems")
engine.add_document( doc_id: 4,  text: "Electric power systems and electric control")
while True:
    print("\n========= MINI SEARCH ENGINE =========")
    print("1. Search Keyword")
    print("2. Exit")
    choice = input("Enter choice: ")
    if choice == "1":
        keyword = input("Enter keyword: ")
        engine.search(keyword)
    elif choice == "2":
        print("Exiting program.")
        break
    else:
        print("Invalid choice.")
```

Output:

```
========= MINI SEARCH ENGINE =========
1. Search Keyword
2. Exit
Enter choice: 1
Enter keyword: Electric

Ranked Search Results:
Document ID: 4 | Relevance Score: 2
Document ID: 2 | Relevance Score: 2
Document ID: 1 | Relevance Score: 1

========= MINI SEARCH ENGINE =========
1. Search Keyword
2. Exit
Enter choice: 1
Enter keyword: power

Ranked Search Results:
Document ID: 4 | Relevance Score: 1

========= MINI SEARCH ENGINE =========
1. Search Keyword
2. Exit
Enter choice: 2
Exiting program.

Process finished with exit code 0
```